arXiv:2611.13176v1 [cs.DC] 9 Nov 2026

SHADOWPREFILL: KEEPING YOUR NEXT MODEL WARM FOR PER-STEP MODEL ROUTING IN AGENTIC LLM SERVING

Haoran Qiu [*1] **Rohan Mahapatra** [*1] **Rodrigo Fonseca** [1] **Mark Russinovich** [2]

ABSTRACT

Per-step model routers send each LLM call of an agent to the cost-optimal model dynamically within each session. However, every model switch discards prefix-cache reuse: KV caches are per model, so the new model must prefill the relevant context before it emits a token. Across representative agentic workloads, including a production coding-agent trace with a mean context of 63K tokens, the prefix hit rate falls from 80–89% on same-model calls to 44–65% after a switch, and a switch call takes 1.5–3.2× as long. We present SHADOWPREFILL, which prefills the session's context on the model the router has not chosen, in capacity that instances serving normal traffic leave idle, so the next switch finds its prefix cached. SHADOWPREFILL harvests the per-iteration SLO slack of requests in flight, protects routed calls, spills shadow KV to host memory, and falls back to a cold switch whenever the shadow is incomplete. With LLMRouter routing 24 model pairs over five agentic benchmark suites, SHADOWPREFILL cuts switch-call latency by 8.5–21.4%, LLM-call latency by 3.7–17.7%, and routed token cost by 22.5–46.1%, while normal traffic keeps its SLO and energy per request rises by 0.4%.

1 INTRODUCTION

Motivation. Agents solve a task through a sequence of LLM calls: plan, call a tool, read the result, edit, test, and repeat. The calls differ widely in difficulty, so a single model is either too weak for the hard steps or too expensive for the easy ones. Routers address this mismatch by selecting a model for each call (Ong et al., 2024; Chen et al., 2023b; Ding et al., 2024), and recent systems make the decision per step inside an agent loop (Zhang et al., 2025; 2026a;c; Paul & Nandy, 2026). Commercial products ship the same pattern: GPT-5 routes between a fast and a reasoning model per request (OpenAI, 2025), and coding agents delegate subtasks to smaller models (Anthropic, 2025). Across representative agentic workloads, an LLMRouter (Feng et al., 2025) policy that routes between two models is 6.6–13.6% more accurate than always using the larger one using less tokens (Section 5.3).

The model switch tax. Agent calls are long and grow: each call resends the session's history plus new tool output. Serving engines make this cheap through prefix caching (Kwon et al., 2023; Gao et al., 2024; Qin et al., 2024): a call whose prompt extends the previous one prefills only the new tokens, and providers bill cached input at about a tenth of the price. KV caches, however, belong to one model. When the router switches, the target model holds no copy of the recent context, or a copy that stops at the last call it served, and must prefill the gap before it emits a token (Liu et al., 2026b). Across representative benchmarks the token-weighted prefix hit rate is 80–89% on same-model calls but 44–65% on switch calls, and a switch call takes 1.5–3.2× as long as a same-model call (Section 2). The switch tax lands on latency, cost, and the flow of the agent loop, and grows with the long, tool-heavy sessions that routing is meant to make affordable.

Idle LLM serving capacity. The model the router will switch to is usually already deployed and serving requests from normal traffic. Such deployments are provisioned for their peak load to avoid SLO violations. In the Azure production LLM inference traces (Stojkovic et al., 2025; Qiu et al., 2025), reserved GPU pools can only be utilized at 50% and 39% when provisioning for the P95 demand of text and multimodal workloads; even at peak, when whole-instance idle time vanishes, decode iterations are memory-bound and run well within their per-token SLO, leaving 69–79% of each iteration's latency budget unused (Section 2.3).

Our Work. We propose **SHADOWPREFILL**, which proactively warms up the prefix cache on the model that the router did not choose by harvesting idle capacity (Figure 1). While the chosen model serves a call, SHADOWPREFILL prefills the session's context on the other model's instance as low-priority work, and streams the call's input and output tokens to it as they appear. Upon model switches, the switch call finds most of its prompt cached; otherwise, shadow KV is evicted first and the normal path is unchanged. Additionally, SHADOWPREFILL must address these challenges:

- **[C1] No free GPUs at peak.** Whole instances are rarely idle when load is high (Section 2.3), which is when switches are most expensive.

[*]Equal contribution. [1]Microsoft Azure Research. [2]Microsoft Azure.

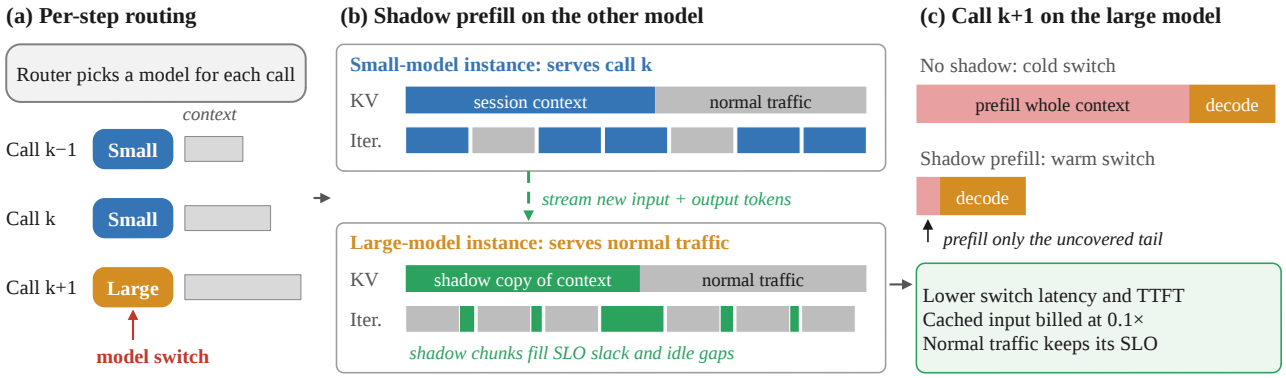

Figure 1. SHADOWPREFILL overview. (a) A per-step router picks a model for every LLM call of an agent session. (b) While the small model serves call k, the large model's instance, which is busy with normal traffic, prefills a shadow copy of the session context in its SLO slack and idle gaps, and receives the new input and output tokens as they appear. (c) When the router switches at call $k+1$, the large model finds the prefix cached and prefills only the uncovered tail from the small model.

- **[C2] Shadow work slows the calls it is meant to speed up.** Shadow chunks share iterations with routed calls and with normal traffic on existing model instances.
- **[C3] Long contexts exceed the KV budget.** A 100K-token coding context occupies a large share of a small model's KV pool and is evicted before it is used.
- **[C4] Most shadow work can be wasted.** Switches occur on 26–33% of calls (Section 2.1), and the router decides only when the call arrives (except for *Plan* mode).
- **[C5] No SLO regression.** A switch must never wait for shadow, and normal traffic must keep its SLO.

SHADOWPREFILL answers these with an SLO-slack harvest controller that fills each iteration up to the latency slack of the requests in flight (C1), routed-step protection that limits stretch on iterations carrying routed calls (C2), a capped shadow KV class with a host-memory spill tier (C3), incremental, streamed, and sticky shadow jobs ordered by switch likelihood (C4), and lowest-priority, iteration-granular shadow work with a cold fallback (C5). It also extends to prefill/decode (PD)-disaggregated clusters.

Results. We implement SHADOWPREFILL on vLLM and evaluate it with LLMRouter (Feng et al., 2025) as the per-step router on 24 model pairs over five agentic benchmarks, including SWE-bench (Jimenez et al., 2024) and the GitHub Copilot coding-agent trace (Liu et al., 2026b), with production LLM inference traffic from Azure (Stojkovic et al., 2025). SHADOWPREFILL reduces switch-call latency by 10.4% on the benchmarks and 14.3% on coding sessions, switch-call TTFT by 44.1% and 47.0%, mean LLM-call latency by 5.0% and 9.7%, and routed token cost by 28.5% and 32.0%. It captures 52% of the latency gain of an oracle that makes switches free, without extra GPUs, while energy per request rises by 0.4% and normal traffic keeps its SLO.

Contributions. In summary, our main contributions are:

- A characterization of per-step routing in agent workloads that quantifies the model switch tax (Section 2).
- SHADOWPREFILL, the first system that warms a routed-to model's prefix cache by harvesting the idle capacity of instances serving normal traffic, with mechanisms for each challenge and a PD-disaggregated variant (Section 3).
- A comprehensive evaluation over 24 model pairs across five agentic benchmarks and traces that demonstrate the benefits from SHADOWPREFILL (Sections 5 and 6).

2 BACKGROUND AND CHARACTERIZATION

2.1 Per-step Routing in Agent Loops

Model routers. A per-step router observes each LLM call of a session and selects one of several deployed models for it. We adopt LLMRouter (Feng et al., 2025) as the router. Its benchmarks evaluate every query on every candidate model, so a routing policy can be replayed exactly; we use its accuracy-per-cost policy, which sends each step to the model with the highest score per dollar, and restrict the candidates to a two-model pair, a small model S and a large model L. Two models is the common deployment of a cheap and a capable tier (OpenAI, 2025; Anthropic, 2025).

Workloads. We study five routed workloads, as summarized in Table 1: xRouteBench (Feng et al., 2026a), FusionBench (Feng et al., 2026b), and Router-R1 (Zhang et al., 2025) are routing benchmarks; we group consecutive queries of a task into sessions and grow the context as an agent loop does, appending each step's input, output, and a tool result (lognormal, median 1K tokens) to the history. GHCP-coding takes its token sizes from one day of the public GitHub Copilot coding-agent trace (Liu et al., 2026a): each session is one user turn of up to 60 agent calls, and each call carries its recorded prompt, which already includes the history and tool outputs. GHCP itself rarely changes model, so

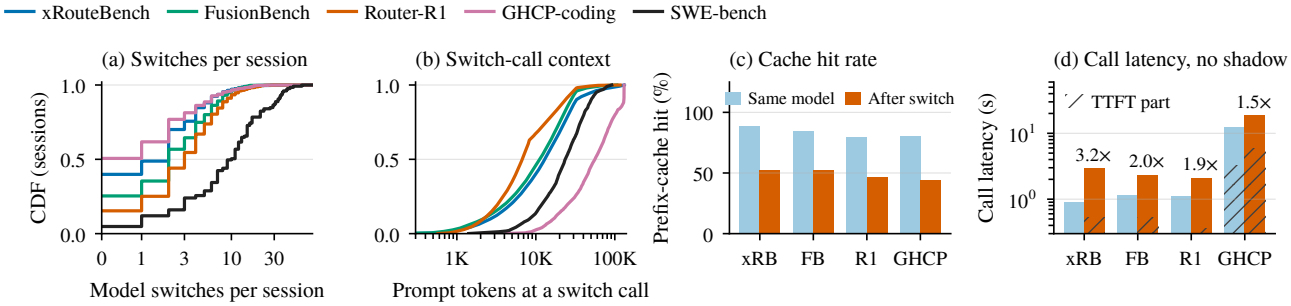

Figure 2. Per-step routing and its switch tax. (a) Model switches per routed session. (b) Prompt length of switch calls. (c) Token-weighted prefix-cache hit rate of same-model and switch calls. (d) Mean call latency (hatched: time to first token); labels are switch-call slowdown.

Table 1. Routed workloads. Prompt sizes in tokens; hit rates are token-weighted over all calls and over switch calls.

Trace	Pairs	Switch	Sw./sess.	Prompt	p99	Hit (sw.)
xRouteBench	5	19%	2.3	19K	113K	80.4 (52)
FusionBench	5	26%	3.1	13K	58K	76.6 (52)
Router-R1	6	33%	4.1	10K	45K	69.2 (46)
GHCP-coding	6	26%	2.0	63K	131K	55.4 (44)
SWE-bench	6	26%	10.9	36K	93K	95.2 (65)

its routing decisions come from routing FusionBench's coding tasks (HumanEval, MBPP) within the same pair. SWE-bench (Jimenez et al., 2024) sessions are SWE-agent trajectories (57 calls on average) whose calls LLMRouter's matrix-factorization router routes between SWE-agent-LM-7B and SWE-agent-LM-32B or Qwen2.5-72B; we replay them on the GPU testbed (Section 6). Per-step routing is frequent: 19–33% of calls switch model and a session switches 2.0–4.1 times on average (Figure 2a); SWE-bench sessions switch 13 times. Coding contexts are long: the mean switch call carries 63K tokens on GHCP and 27K on SWE-bench, 3–6× and 1.3–2.6× the routing benchmarks (Figure 2b). The router is 6.6–13.6 points more accurate than always using the pair's more expensive model, because the two models are right on different queries, yet it saves only 28–44% of that model's token cost, partly because every switch forfeits the cached-input discount (Section 5.3).

2.2 The Model Switch Tax

Prefix caching is per model. Serving engines keep the KV of recent prompts and reuse it when a new prompt shares a prefix (Kwon et al., 2023; Gao et al., 2024; Qin et al., 2024). In an agent loop each call extends the previous one, so a session that stays on one model prefills only the new tool output and input: same-model calls hit 80–89% of their prompt tokens (Figure 2c). KV is a function of the model's weights, so the cache cannot be shared across models. A switch call finds either nothing on the target or a copy that ends where the session last left that model, and its hit rate falls to 44–65%. Same observation is made in a study on Github Copilot model switch behaviors (Liu et al., 2026b).

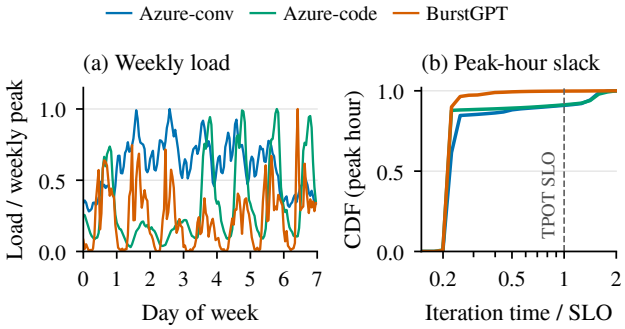

Figure 3. Idle capacity of LLM deployments. (a) Load over one week, relative to the trace's peak. (b) Iteration time relative to the TPOT SLO at the peak hour. Results are for Qwen2.5-7B (one H100 GPU per instance) and Llama-3.3-70B (eight H100 GPUs per instance), with four instances per pool and target utilization $\rho = 0.7$.

The tax is paid in latency, cost, and flow. The lost prefix must be prefilled before the first token. A switch call takes 3.2× as long as a same-model call on xRouteBench and 1.5× on GHCP-coding (Figure 2d); time to first token (TTFT), the stall a user or agent loop waits through before anything streams, accounts for most of the difference. Providers bill cached input at about a tenth of the uncached price, so every lost hit also raises the token consumption for the end user, and the extra prefill occupies GPUs that serve other users. Switch-aware routers (Zhang et al., 2026a;c) avoid switches and give up accuracy; SHADOWPREFILL lowers their price.

2.3 Idle Capacity in Production Deployments

Deployments are provisioned for peak. The model a session switches to is already deployed and busy serving other users' requests, which we call normal traffic. Clusters are typically sized for the peak hour to reserve buffer for bursts and avoid SLO violations, so off-peak hours leave capacity unused. We replay one week of the Azure LLM inference traces for conversation and code (Stojkovic et al., 2025) and BurstGPT (Wang et al., 2025) on a pool provisioned for its peak (Figure 3a). Load swings by 3–10× within a day. Instances serving code traffic are completely idle for 72% of the median hour and 85% of the trough hour, but only 8%

of the peak hour, and conversation traffic, which is steadier, leaves no whole-instance idle time in any window.

Iterations have SLO slack even at peak. Whole-instance idleness disappears exactly when switches are slowest. A finer-grained resource remains: a continuous-batching iteration is dominated by memory-bound decode and finishes well within the time-per-output-token (TPOT) SLO of the requests it serves. At the peak hour, the median iteration uses about a quarter of the TPOT SLO, and 69–79% of the per-iteration latency budget is unused (Figure 3b). A prefill chunk added to such an iteration delays every decode in it (Agrawal et al., 2024b), but as long as the iteration stays within the slack of each request in flight, no request misses its SLO. This slack is the capacity SHADOWPREFILL harvests.

3 DESIGN

SHADOWPREFILL sits between the router and the serving engines of a model pair. For every routed call it creates or extends a *shadow job* on the model the router did not choose, and each engine runs shadow jobs as a lowest-priority work class. This section states the goal and the mechanisms to address the challenges: Shadow jobs (**[C4]**, Section 3.1), harvesting (**[C1]**, **[C2]**, Section 3.2), Shadow KV management (**[C3]**, Section 3.3), and safety (**[C5]**, Section 3.4).

3.1 Goal and Shadow Jobs

Goal. When a session switches from model A to model B at call $k{+}1$, the prompt of that call consists of the session's history h_k (all prompts and outputs through call k) plus the new input. SHADOWPREFILL aims to have h_k resident in B's prefix cache when call $k{+}1$ arrives, so that the switch call prefills only the new input, as a same-model call does. It may spend only capacity that would otherwise go unused (by harvesting idle capacity), and it must never delay a switch call or cause normal traffic to miss its SLO.

Incremental shadow jobs. A shadow job for session s on model B records the prefix of h that B already holds and the tokens still missing. When call k arrives at A, the manager appends call k's new input to the job immediately, and appends A's output tokens once A generates them (output streaming). B therefore never re-prefills what it already has: after a switch back from B to A and forward again, the job covers only the delta. The shadow always trails the routed model by at least the output of the current call; the switch call prefills that tail itself as it would do at each switch.

Sticky model placement. Each agentic session keeps a sticky instance per model (Agarwal et al., 2026), and the shadow job runs on the sticky instance of the other model. The router sends each call to the instance of the chosen model with the lowest estimated prefill completion time, counting only uncached tokens. A switch call therefore lands on the instance that holds its shadow, unless that instance is so busy that a cold prefill elsewhere would finish first.

Ordering by switch likelihood (C4). The engine runs pending shadow jobs in order of their predicted switch probability, estimated per task and current model from past sessions, so the capacity goes first to the sessions most likely to switch. Gating jobs below a probability threshold reduces waste further but loses switches the predictor misses; we evaluate it as the Predictive variant in Section 5.

3.2 Harvesting Capacity

Where shadow work runs. Each iteration of a continuous-batching engine schedules, in priority order, all running decodes, in-flight prefill chunks, and admissions from the waiting queue, under a token budget. SHADOWPREFILL adds shadow chunks last, and only if the waiting queue is empty and no admission was blocked by memory.

Idle instances. When an instance has no other work, it runs shadow-only iterations. Each such iteration is capped at β times the TPOT SLO ($\beta{=}0.5$), which bounds how long a newly arriving request waits behind it.

SLO-slack controller (C1). At peak, instances are rarely idle, so SHADOWPREFILL piggybacks shadow chunks on normal iterations, sized by the latency slack of the requests in flight. The controller uses only what a production scheduler observes: arrival times, first-token times, and iteration counts. A decoding request r that received its first token at f_r and has completed k_r iterations meets a TPOT SLO T_r with margin m if $t - f_r \le (1{-}m)T_r k_r$, so the next iteration may last up to

$$\sigma_r = (1{-}m)T_r(k_r{+}1) - (t - f_r). \tag{1}$$

A prefilling request has slack $(1{-}m)F_r - (t - a_r) - \hat{p}_r$, where F_r is its TTFT SLO, a_r its arrival, and $\hat{p}_r$ the predicted time of its remaining chunks. The iteration budget is the smallest slack in the batch, capped at βT; the controller then adds shadow tokens while the latency model predicts the iteration to stay within budget (Algorithm 1). Requests that are ahead of their SLO, which at the median iteration is most of them (Figure 3b), lend their slack to shadow work; requests close to their deadline block it.

Routed-step protection (C2). Routed calls have no SLO slack to lend: any stretch of an iteration that carries one adds directly to the agent's latency, including that of switch calls the shadow is meant to speed up. When a routed call is prefill/decoding on the instance, the controller instead spends a small credit that accrues at δ times the duration of memory-bound, decode-only iterations ($\delta{=}0.025$), capped by the budget. The long-run slowdown of routed calls is therefore at most δ, while iterations that carry only normal

Algorithm 1 Shadow tokens added to one iteration.

Require: batch $\mathcal{B}$ of decodes and prefill chunks; shadow queue Q
if waiting queue nonempty **or** admission blocked **then**
  return 0
else if $\mathcal{B}=\emptyset$ **then**
  $b \leftarrow \beta T$ idle instance
else if $\mathcal{B}$ holds a routed call **then**
  $b \leftarrow \min(credit, \beta T)$ C2
else
  $b \leftarrow \min(\beta T, \min_{r\in\mathcal{B}} \sigma_r)$ C1
end if
$n \leftarrow \max\{n : \hat{t}(\mathcal{B}+n \text{ shadow tokens}) \le b\}$
$n \leftarrow \min(n, \text{free shadow KV})$
if $n < n_{\min}$ **then**
  return 0
end if
take n tokens from jobs in Q by switch probability
return n

traffic harvest their full slack. Without the SLO controller, Harvest uses the same credit on every busy iteration (δ=0.1).

Sensitivity studies on parameters are in Sections C and 5.5.

3.3 Shadow KV Management

A capped, evict-first class (C3). Shadow KV is useful only if it survives until the switch, but it must not displace the cache entries that make same-model calls fast. Shadow entries form their own cache class, limited to 25% of the KV pool. A shadow chunk may use free blocks or blocks of cache entries idle for more than 5 s. Eviction removes shadow entries before normal ones and is LRU within each class. Configuration sensitivity studies are in Section 5.5.

Host-memory spill tier. SHADOWPREFILL spills evicted session entries, shadow and normal, to host DRAM and reloads them over PCIe when a call hits them, as KV offloading systems do for a single model (Gao et al., 2024; Liu et al., 2025). Reloading a prefix is far cheaper than recomputing it, so the tier keeps shadow work from being lost to HBM pressure; it also helps same-model calls whose prefixes were evicted, which we evaluate separately as the Host Tier variant (Section 5). Host tier is needed especially for long-context agents (e.g., coding (Liu et al., 2026b)): Gemma-7B uses full multi-head attention and stores 458 KB of KV per token, so a single 100K-token agent context fills most of its H100's KV pool, and its shadow is evicted long before the switch.

3.4 Safety and Fallback

No regression by construction (C5). Shadow work never blocks a routed call or a normal request: it runs only when nothing is waiting, it is bounded per iteration by the budget above, and it is preempted at iteration granularity, so an arriving request waits at most one capped iteration. A switch call never waits for a shadow. If its shadow is partial, it uses the cached prefix and prefills the uncovered tail; if the shadow is absent or was evicted, it runs exactly as without SHADOWPREFILL. The price of a wrong prediction is the harvested capacity it consumed, plus the energy of that computation (Section 5.4).

3.5 Prefill/Decode Disaggregation Extension

PD disaggregation runs prefill and decode on separate instances and moves the KV between them (Patel et al., 2024; Qin et al., 2024). A switch there costs both a cold prefill on the target's prefill (P) instances and a larger KV transfer to its decode (D) instance. SHADOWPREFILL runs shadow jobs on the session's sticky P instance of the other model when its queue is empty and background requests have TTFT slack, in iterations of at most 0.1 s. P instances are provisioned for prefill-heavy peaks and idle often, so shadow work finds more capacity than in a colocated cluster. The sticky D instance of the other model then pulls the shadow KV in 4K-token pushes whenever its inbound link carries no normal transfer, into free decode memory that is reclaimed first. At the switch, the P instance prefills only the tail and the D instance receives only the tokens it lacks.

4 IMPLEMENTATION

Router and sessions. We build routed sessions from LLM-Router's benchmark matrices (Feng et al., 2025) by replaying the accuracy-per-cost policy over each model pair (Table 3). A session builder grows each step's prompt from the session history, the dataset input, and optionally sampled tool output; it compacts the history into a 1K-token summary when the context reaches the model's window.

Serving simulator. We implement on top of Vidur (Agrawal et al., 2024a), a high-fidelity LLM inference simulator, with additionally 2,700 lines of Python. The resulting simulator is a discrete-event model of a cluster of continuous-batching instances. Each iteration's latency is grounded by a per-layer profiled model of kernels (GEMMs, prefill attention, decode attention, and tensor-parallel all-reduces) fitted to each model's H100 operator profiles. Model instances run chunked prefill with a default 2,048-token budget and up to 256 sequences, reserve KV cache at admission, and maintain a prefix cache of session contexts and finished requests.

vLLM prototype. We build a prototype to validate the core mechanism of SHADOWPREFILL in a real serving system. For each model pair, we run two vLLM (Kwon et al., 2023) instances, one per model, with prefix caching and priority scheduling enabled. A router selects the model for each call in an agentic session, and a router-side manager implements a request-level version of Harvest. After a call completes on one instance, the manager submits its prompt and response to the other instance as low-priority shadow work, allowing

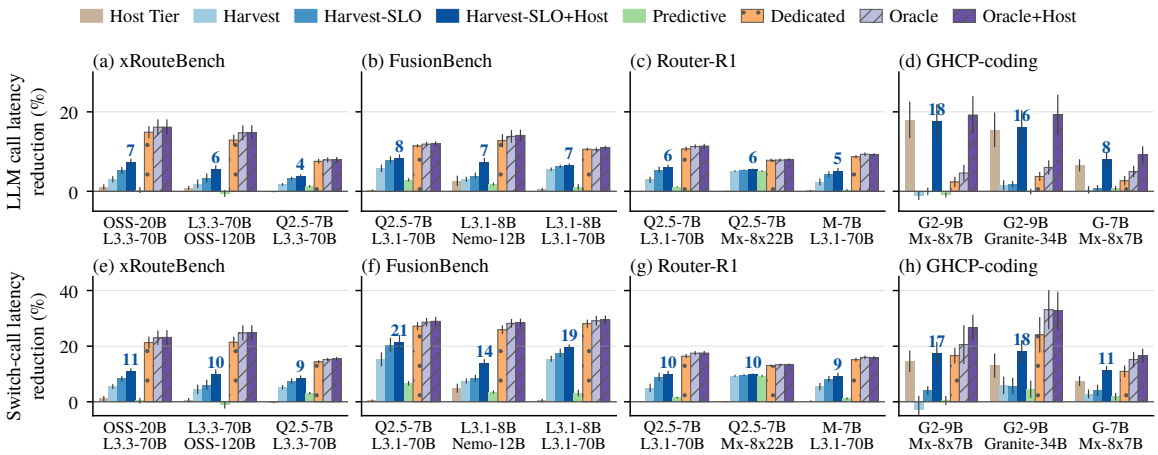

Figure 4. Reduction in time-to-last-token latency of (a–d) all LLM calls and (e–h) switch calls relative to No Shadow for each trace. Labels indicate the improvements from Harvest-SLO+Host. Error bars: ±1 SE over 3 seeds × 3 windows.

that model to compute and retain its own KV for reuse after a subsequent model switch. For learned model routing, we follow LLMRouter's matrix-factorization methodology (Feng et al., 2025) and retrain a router for each model pair. The implementation is approximately 840 lines of Python code and require no modifications to vLLM source code. Further details are in Section D.1.

5 EVALUATION

We evaluate SHADOWPREFILL through trace-driven simulation in this section and GPU testbed experiments in Section 6.

5.1 Setup

Model pairs and benchmarks. We simulate 22 model pairs: 5 on xRouteBench, 5 on FusionBench, 6 on Router-R1, and 6 on GHCP-coding (Table 1; full list in Table 3); the GPU testbed adds two SWE-bench pairs (Section 6). The simulated pairs span dense and MoE models from 7B to 141B parameters, same- and cross-family pairs, and the Qwen, Llama, Mistral, Gemma, GPT-OSS, and Granite families. The appendix shows all model pairs (Table 3 and Table 4).

Production traffic. Each model runs on 4 instances, each using the smallest tensor-parallel degree among 1, 2, 4, and 8 H100 GPUs at which weights take at most half the HBM. Each instance serves normal traffic replayed from the Azure LLM inference conversation trace (Stojkovic et al., 2025) by default. Pools are provisioned for the trace's peak hour: the normal-traffic rate is ρ=0.7 of the capacity at which 90% of requests meet both SLOs at peak, and we run three one-hour windows (peak, median, and trough) at their relative trace load. The SLOs are TTFT $\leq \max(1\,\text{s}, 5\times$ isolated prefill time) and TPOT $\leq \max(25\text{ms}, 5\times$ batch-1 decode time).

Table 2. Approaches compared.

Approach	Behavior
No Shadow	Cold switch; the baseline.
Host Tier	No Shadow with the host-DRAM KV tier.
Harvest	Shadow jobs under a fixed credit (δ=0.1).
Harvest-SLO	Shadow jobs under the SLO-slack controller.
Harvest-SLO+Host	Harvest-SLO with the host tier.
Predictive	Harvest, gated on switch probability $\geq$0.25.
Dedicated	One extra instance per model runs all shadow work (+25% GPUs).
Eager	Shadow work as normal-priority requests.
Oracle (+Host)	Switch call's prefix cached for free.

Measurement. Each run measures 600 s (1,800 s for GHCP-coding) after a 120 s warmup and drains until every measured session finishes. We run 3 seeds × 3 windows per configuration and compare each approach with No Shadow on the same seed, window, and pair, over the calls that complete under every approach; error bars are ±1 standard error over these runs. Token cost uses each benchmark's per-model prices, with cached input billed at 10% of uncached.

Baselines, ablations, and oracle. Table 2 lists the different system configurations. Host Tier isolates what the host tier alone does for same-model calls; Harvest and Harvest-SLO isolate the two controllers; Harvest-SLO+Host is the complete design. Dedicated spends extra GPUs instead of harvesting, and Eager uses the engine's normal request path, as prior cache-warming systems do for a single model (Zhang et al., 2026b). Oracle and Oracle+Host bound what any warming scheme can achieve without and with the host tier. Figures omit Eager, which significantly slows LLM calls.

5.2 Latency and Cost Comparison

Switch calls. SHADOWPREFILL improves the switch tax directly (Figure 4e–h). Harvest-SLO+Host cuts switch-call latency by 10.4% on average over the 16 benchmark pairs

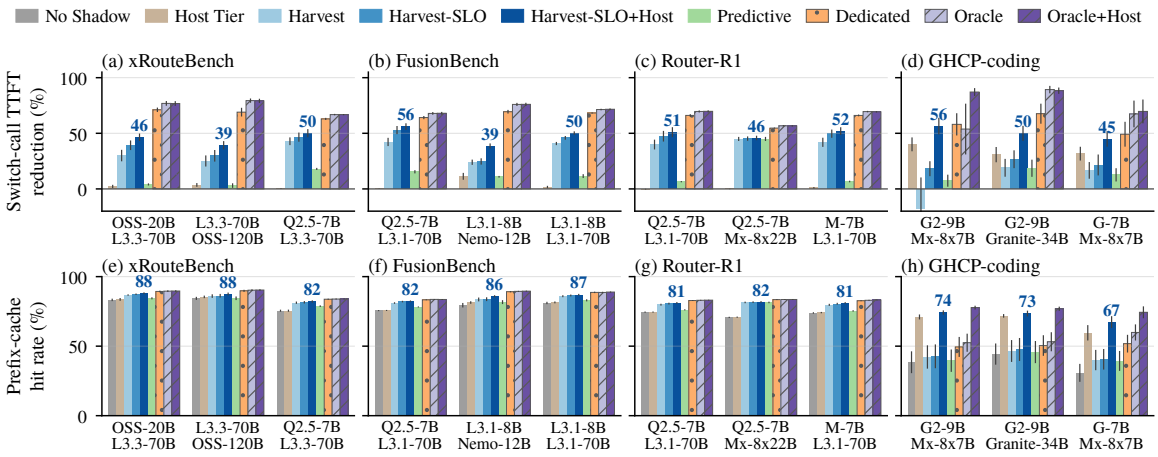

Figure 5. (a–d) Reduction in switch-call TTFT relative to No Shadow. (e–h) Token-weighted prefix-cache hit rate of all LLM calls. Labels indicate the percentage improvement or cache hit performance from Harvest-SLO+Host.

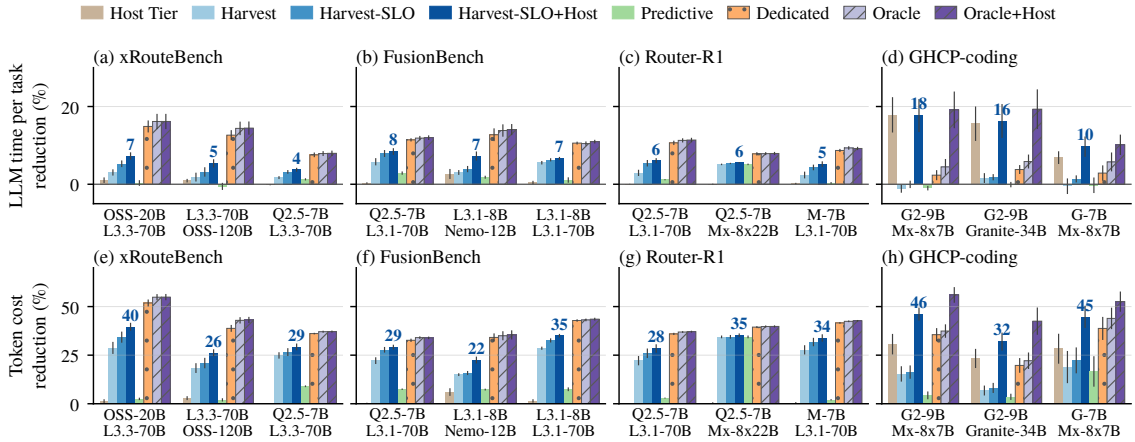

Figure 6. (a–d) Reduction in LLM time per task (sum of a session's call latencies) and (e–h) reduction in routed token cost, relative to No Shadow. Labels indicate the percentage improvement from Harvest-SLO+Host.

and by 14.3% over the 6 GHCP-coding pairs, and by up to 21.4% on a single pair. Oracle bounds the achievable reduction at 19.3% and 27.4% (Oracle+Host on GHCP), and Harvest-SLO+Host reaches 54% and 52% of it without extra GPUs. On GHCP-coding a cold switch call takes 19.0 s, and SHADOWPREFILL saves 3.3 s of it. The improvement comes mostly from the time to generate the first token: switch-call TTFT falls by 44.1% and 47.0% (Figure 5a–d).

All LLM calls. Switch calls are 19–33% of all LLM calls, so the mean latency of all LLM calls falls by 5.0% on the benchmarks and 9.7% on GHCP-coding (Figure 4a–d), against Oracle bounds of 9.6% and 13.8%.

Cache hit rate. Figure 5e–h shows the benefits of SHADOWPREFILL on cache reuse. Harvest-SLO+Host improves the token-weighted hit rate of all calls from 75.0% to 82.1% on the benchmarks and from 55.4% to 76.3% on GHCP-coding, close to Oracle+Host (84.4% and 81.3%).

Ablation on each mechanism. The fixed-credit Harvest controller reaches 6.9% on the benchmarks: it spends the same small credit on every busy iteration, whatever the slack of the requests in it. The SLO-slack controller lifts this to 8.8%, and the host tier to 10.4%. On GHCP-coding, Harvest-SLO reaches only 4.3%, because a 63K-token shadow on a small model's KV pool is evicted before the switch; the host tier keeps it, and Harvest-SLO+Host reaches 14.3%. Host Tier alone barely helps the benchmarks (0.4% LLM-call latency), whose contexts fit in HBM. On GHCP it recovers evicted prefixes of same-model calls too and cuts LLM-call latency by 8.1%, but shadow work on top of it is what fixes switch calls: TTFT falls by 27.5% with Host Tier and 47.0% with Harvest-SLO+Host, and token cost by 17.8% and 32.0%. Predictive saves less than Harvest (3.2% *vs.* 6.9%): the router decides per call, and a gate on predicted switch probability skips many switches that do happen. Dedicated approaches Oracle (18.0%), but needs 25% more GPUs and 11.7% more energy per request (Figure 8). Eager (not shown) issues

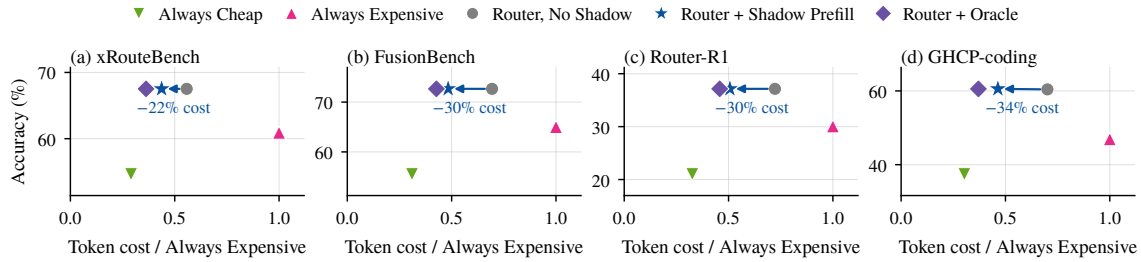

Figure 7. Accuracy versus routed token cost, relative to Always Expensive, averaged over each trace's model pairs. Arrows mark the cost reduction from SHADOWPREFILL (Harvest-SLO+Host) at equal accuracy.

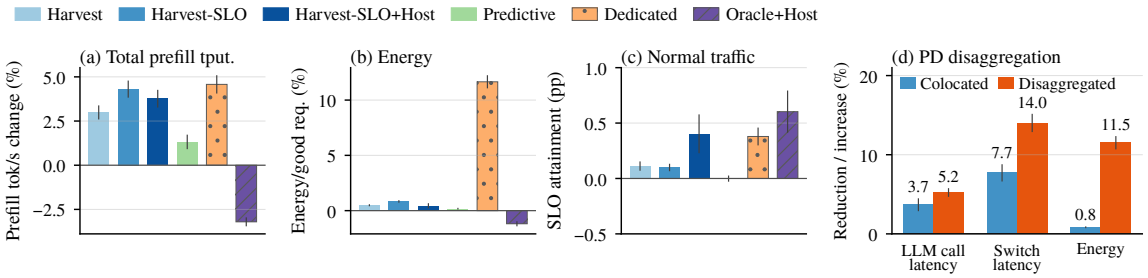

Figure 8. Overheads over all 22 simulated pairs, relative to No Shadow. (a) Total prefill throughput, including shadow tokens. (b) Energy per request meeting its SLO. (c) Normal-traffic SLO attainment. (d) Harvest-SLO on colocated and P/D-disaggregated clusters.

shadow work as ordinary requests and slows LLM calls by 7.0% on the benchmarks and 3.2% on GHCP-coding.

Task completion time. An agent task issues its calls in sequence, so the LLM time per task falls with the per-call latency (Figure 6a–d): by 4.9% across the agentic benchmarks and by 10.0% on GHCP-coding traces, and by up to 17.7% on the model pair of Gemma-2-9B and Mixtral-8x7B.

User-side token cost. Every token a switch call finds cached is billed at a tenth of the price (Anthropic, 2025), so SHADOWPREFILL cuts the routed token bill by 28.5% on the benchmarks and 32.0% on GHCP-coding, and by up to 46.1% (Figure 6e–h) due to its longer prefix.

5.3 Accuracy and Cost of Routing

SHADOWPREFILL changes the cost of routing without changing its decisions. Figure 7 places the router, with and without SHADOWPREFILL, against Always Cheap and Always Expensive, which send every call to the pair's cheaper or pricier model by price. The router is 6.6–13.6 points more accurate than Always Expensive, since the two models answer different queries correctly (Feng et al., 2025). Without SHADOWPREFILL it saves 28–44% of Always Expensive's cost; with SHADOWPREFILL it saves 49–56%, cutting the router's own bill by 22–34% at the same accuracy. On GHCP-coding, routing with SHADOWPREFILL is 13.6% more accurate than Always Expensive at 54% lower cost.

5.4 Overheads and Disaggregation

Token throughput. SHADOWPREFILL moves prefill work from the critical path into harvested idle capacity: under Harvest-SLO+Host, total prefill throughput, including shadow tokens, rises by 3.8% (Figure 8a), while the prefill throughput of routed and normal requests alone changes by -2.4%, because switch calls find their prefix cached. Decode throughput is unchanged.

Energy and normal traffic. Shadow work runs on GPUs that are powered anyway, in slack that is mostly memory-bound, so energy per good request rises by only 0.4% under Harvest-SLO+Host and 0.8% under Harvest-SLO, compared with 11.7% for Dedicated (Figure 8b). Normal-traffic SLO attainment does not drop under any harvesting approach; it rises by 0.10–0.40 points (Figure 8c), because switch calls that hit the cache leave the instances sooner.

Prefill/Decode disaggregation. On a disaggregated cluster, each model's 4 instances split into prefill and decode instances, provisioned per trace for the same SLOs. Prefill instances idle for over half of each run, so Harvest-SLO finds more capacity: across all pairs it cuts LLM-call latency by 5.2% and switch-call latency by 14.0%, compared with 3.7% and 7.7% colocated (Figure 8d), and reaches 78% of the disaggregated oracle's gain on the benchmarks. The price is energy: colocated shadow work rides on iterations that run anyway, whereas disaggregated shadow work wakes otherwise-idle prefill GPUs, raising energy per good request by 11.5% (0.8% colocated).

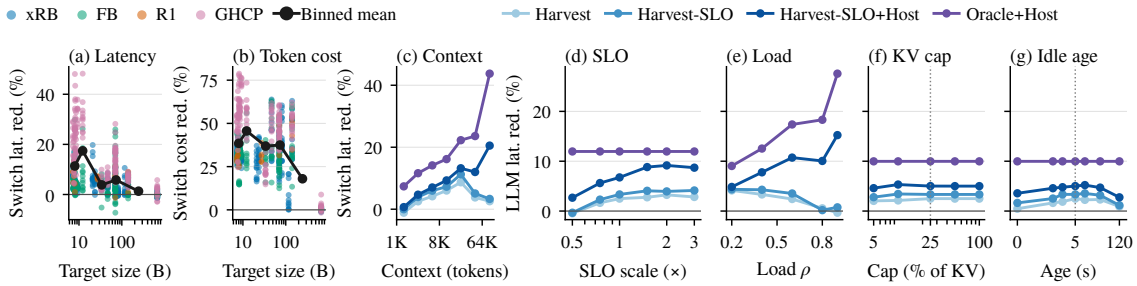

Figure 9. Where the benefit comes from. Switch-call (a) latency and (b) cost reduction of Harvest-SLO+Host versus the size of the target model (points: pair and target; line: call-weighted bin mean). (c) Switch-call latency reduction versus context length. LLM-call latency reduction versus (d) SLO scale, (e) load ρ, (f) shadow KV cap, and (g) cold-entry age (dotted: default).

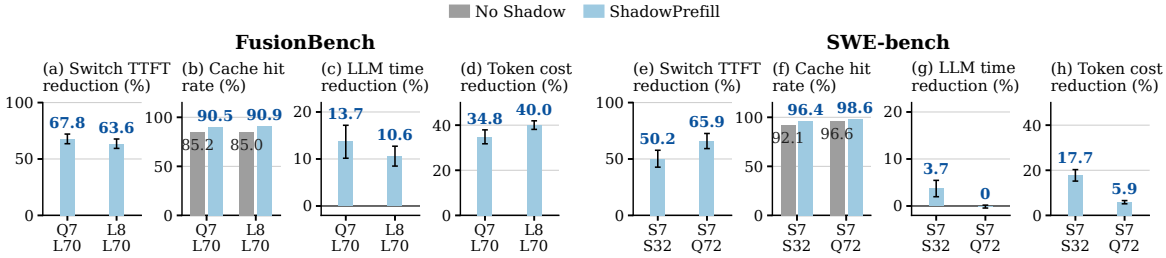

Figure 10. Switch-call TTFT reduction, prefix-cache hit rate, LLM time per task reduction, and token cost reduction relative to No Shadow.

5.5 Where the Benefit Comes From

Target model size. Figure 9a–b breaks the switch-call benefit down by the model a switch goes to, over 283 screened pairs. Latency falls by 12% for targets under 15B, 4% for 15–50B, and 3% above 50B. Shadow Prefill raises the switch-call hit rate by 24–28 points at every size up to 141B, but the prefill it removes is 26% of a cold switch call on small targets and 12–13% on larger ones, whose decode dominates. DeepSeek-R1 (671B) gains 0.2 points: its long reasoning outputs (355 tokens on average) leave its instances little slack. Cost falls by 40%, 37%, and 24% in the same bins, as output tokens take a growing share of the bill.

Context length. The longer the context before a switch, the more a cold switch costs and the more SHADOWPREFILL saves (Figure 9c). Harvest-SLO+Host cuts switch-call latency by 0.7% for contexts below 2K tokens and by 20.5% beyond 64K, and Oracle+Host by up to 43.7%. Without the host tier, Harvest-SLO falls back above 32K, because long shadows are evicted from HBM.

SLO strictness. Tighter SLOs leave less slack to harvest (Figure 9d; d–g average one pair per trace). At half the default SLOs, with provisioning fixed, Harvest-SLO+Host still cuts LLM-call latency by 2.7%; at the default it cuts 6.7%, and at 2–3× looser SLOs about 8.7%.

Load. As normal traffic grows, idle capacity shrinks and Harvest-SLO's gain falls from 4.4% at ρ=0.2 to 0.7% at ρ=0.9 (Figure 9e). At the same time, cold switch calls queue longer and pools evict more, so the host tier matters more and Harvest-SLO+Host's gain grows to 15.2%.

Shadow KV cap and cold-entry age. The shadow class rarely fills its 25% cap: shadow KV averages 1.7% of the pool, because shadows enter only free memory and cold entries and are evicted first. Raising the cap to the whole pool leaves Harvest-SLO+Host at 5.0% (Figure 9f); shrinking it to 5% lowers Harvest-SLO+Host only to 4.6%. The cold-entry age sets which cached entries a shadow may displace (Figure 9g). At 0 s, shadows displace entries that are about to be reused, including other shadows: shadow work grows 2.1× and Harvest-SLO+Host drops to 3.6%. At 120 s, shadows fit only into free memory, the switch-call hit rate falls from 72% to 54%, and Harvest-SLO+Host drops to 2.8%. Any age between 2 and 10 s gives 4.8–5.2%.

Section C further varies the router, the controller parameters, the host tier, the cluster scale, the normal-traffic trace, the P/D configuration, and the chunk sizes in sensitivity study.

6 TESTBED VALIDATION

We complement the simulation study in Section 5 with a vLLM-based prototype (Kwon et al., 2023), evaluating SHADOWPREFILL (Harvest) on a 16-H100 GPU testbed.

Setup. We evaluate two FusionBench (Feng et al., 2026b) model pairs that are also used in simulation: Qwen2.5-7B and Llama-3.1-8B, each paired with Llama-3.1-70B. To additionally evaluate learned routing and task accuracy, we use SWE-bench Verified (Jimenez et al., 2024), since the GHCP-coding trace provides token lengths but not the prompt text needed for these experiments. We pair SWE-agent-LM-7B with SWE-agent-LM-32B, both fine-tuned for

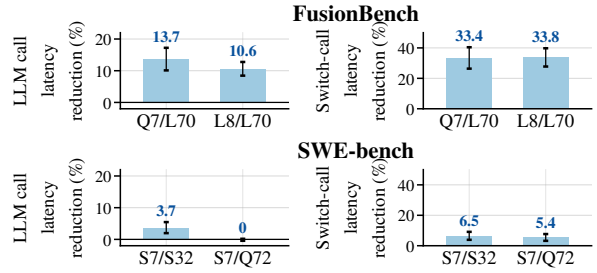

Figure 11. LLM-call and switch-call latency reductions.

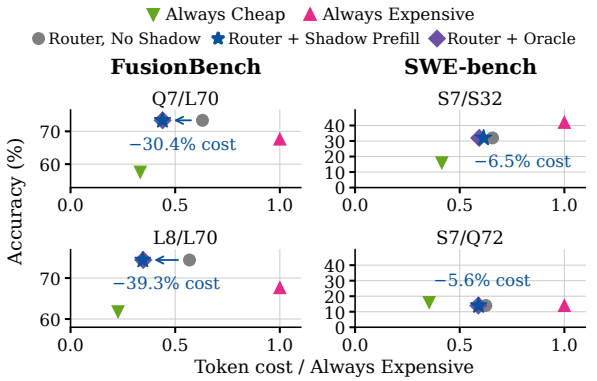

Figure 12. Accuracy versus normalized token cost. SWE-bench shadow costs use replay-measured cache reuse.

software-engineering tasks, and include SWE-agent-LM-7B paired with Qwen2.5-72B-Instruct as a second configuration. We measure task accuracy on a fixed random sample of 50 tasks from the 500-task SWE-bench Verified set, then replay recorded agent calls and routing decisions for serving-performance comparisons. Sections D.2 and D.3 provide the routing and accuracy-evaluation details. To evaluate serving performance separately, we replay recorded calls and routing decisions with and without shadow prefill (Harvest).

Each model instance also serves normal traffic with Poisson arrivals and Azure-derived request lengths, following the methodology in Section 5. We set the normal-traffic arrival rate to ρ=0.3 of each instance. Section D.1 specifies the measurement windows, call matching, and uncertainty estimates.

LLM time per task. Figure 10 shows that SHADOWPREFILL reduces switch-call TTFT by 61.9% on average through prefix reuse. Both FusionBench pairs also achieve lower LLM-call latency, consistent with simulation (Figure 11). The reduction in LLM time per task is smaller because shadow prefill accelerates switch-call prefill, not decoding. It averages 12.1% on FusionBench and 7.0% across all pairs. S7/Q72 illustrates this distinction: only 13% of calls switch models, and decoding dominates execution. A 65.9% switch-call TTFT reduction therefore leaves its LLM time per task approximately unchanged (Table 5).

Accuracy and token cost. Figure 12 shows that SHADOWPREFILL lowers estimated token cost by 20.4% on average.

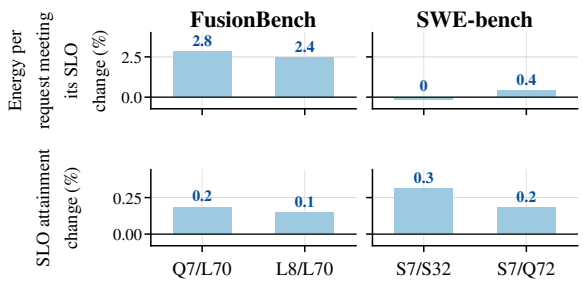

Figure 13. Energy and traffic SLO changes relative to No Shadow.

The cost reduction holds on both benchmarks, consistent with simulation, even when decoding limits task-time gains. Accuracy, in contrast, depends on the selected model pair. On SWE-bench, routing with the agent-tuned S32 resolves 32% of tasks, compared with 14% using the larger, general instruction-tuned Q72. Thus, larger model sizes alone do not ensure higher agent accuracy (more details in Section D.3).

Energy and normal traffic. Figure 13 shows a 1.4% average increase in energy per SLO-compliant request relative to No Shadow. Normal-traffic SLO attainment improves slightly on every pair. Thus, at the evaluated load, the TTFT and cost benefits require little additional energy without compromising normal-traffic SLO attainment.

7 RELATED WORK

Prefix caching and KV management. vLLM, SGLang, and CachedAttention reuse KV across requests that share a prefix (Kwon et al., 2023; Zheng et al., 2024; Gao et al., 2024); Mooncake, Pensieve, and LMCache extend the cache to host memory and remote storage (Qin et al., 2024; Yu et al., 2025; Liu et al., 2025). InferCept, KVFlow, and Continuum keep or prefetch a program's KV across tool calls (Abhyankar et al., 2024; Pan et al., 2025; Li et al., 2025). *All of them manage the cache of the model that produced it;* SHADOWPREFILL *builds a cache on a model that has not served the session.* Across models, recent works translate one model's KV into another's (Liu et al., 2024; Fu et al., 2026; Dery et al., 2026; Heo et al., 2026), and ICaRus and PrefillShare train models to share a cache (Woo et al., 2026a;b). *They require retraining and approximate the target's KV while* SHADOWPREFILL *computes the exact KV (lossless).*

Harvesting and co-location. Harvest VMs and GPU co-location systems run low-priority work in spare capacity (Ambati et al., 2020; Han et al., 2022; Strati et al., 2024). ConServe co-serves offline batch jobs with online inference (Qiao et al., 2026). Sarathi-Serve bounds iteration time with chunked prefill (Agrawal et al., 2024c). SHADOWPREFILL *applies LLM inference harvesting uniquely to a new kind of work in agentic model routing.*

8 CONCLUSION

We characterized the switch tax in per-step LLM routing and presented SHADOWPREFILL, a system that exploits idle GPU capacity to warm prefix caches on routed-to models without additional GPUs or compromising normal-traffic SLOs. Across 24 model pairs and five agentic benchmarks and traces, SHADOWPREFILL reduces switch-call latency by 8.5–21.4% and routed token cost by 22.5–46.1%.

REFERENCES

Abhyankar, R., He, Z., Srivatsa, V., Zhang, H., and Zhang, Y. InferCept: Efficient intercept support for augmented large language model inference. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, 2024. URL https://proceedings.mlr.press/v235/abhyankar24a.html.

Agarwal, S., Hu, B., Mao, A., Akella, A., and Venkataraman, S. SYMPHONY: Enabling compute-memory disaggregation in LLM serving systems. In *23rd USENIX Symposium on Networked Systems Design and Implementation (NSDI 26)*, pp. 2027–2041, 2026.

Agrawal, A., Kedia, N., Mohan, J., Panwar, A., Kwatra, N., Gulavani, B. S., Ramjee, R., and Tumanov, A. Vidur: A large-scale simulation framework for LLM inference. In *Proceedings of Machine Learning and Systems (MLSys)*, volume 6, 2024a. URL https://proceedings.mlsys.org/paper_files/paper/2024/hash/b74a8de47d2b3c928360e0a011f48351-Abstract-Conference.html.

Agrawal, A., Kedia, N., Panwar, A., Mohan, J., Kwatra, N., Gulavani, B., Tumanov, A., and Ramjee, R. Taming throughput-latency tradeoff in LLM inference with Sarathi-Serve. In *18th USENIX symposium on operating systems design and implementation (OSDI 24)*, pp. 117–134, 2024b.

Agrawal, A., Kedia, N., Panwar, A., Mohan, J., Kwatra, N., Gulavani, B. S., Tumanov, A., and Ramjee, R. Taming throughput-latency tradeoff in LLM inference with Sarathi-Serve. In *OSDI*, 2024c.

Ambati, P., Goiri, Í., Frujeri, F., Gun, A., Wang, K., Dolan, B., Corell, B., Pasupuleti, S., et al. Providing SLOs for resource-harvesting VMs in cloud platforms. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2020. URL https://www.usenix.org/conference/osdi20/presentation/ambati.

Anthropic. Claude Code documentation: Prompt caching and model configuration. https://docs.anthropic.com/, 2025.

Chen, C., Borgeaud, S., Irving, G., Lespiau, J.-B., Sifre, L., and Jumper, J. Accelerating large language model decoding with speculative sampling. *arXiv preprint arXiv:2302.01318*, 2023a.

Chen, L., Zaharia, M., and Zou, J. FrugalGPT: How to use large language models while reducing cost and improving performance. *arXiv preprint arXiv:2305.05176*, 2023b.

Dery, L. M., Yahav, Z., Prior, H., Feng, Q., Shen, J., and Szlam, A. Latent space communication via K-V cache alignment. *arXiv preprint arXiv:2601.06123*, 2026.

Ding, D., Mallick, A., Wang, C., Sim, R., Mukherjee, S., Ruhle, V., Lakshmanan, L. V. S., and Awadallah, A. H. Hybrid LLM: Cost-efficient and quality-aware query routing. *arXiv preprint arXiv:2404.14618*, 2024.

Feng, T., Yu, F., Zhang, H., Dai, Z., Yuan, L., Lei, Z., Zhang, W., Zhu, K., et al. LLMRouter: An open-source library for LLM routing. https://github.com/ulab-uiuc/LLMRouter, 2025. Open-source library from ulab-uiuc (UIUC). Companion technical report: Feng et al., "LLMRouter: Unified Infrastructure for Developing, Evaluating, and Deploying LLM Routers," arXiv:2608.06867 (2026).

Feng, T., Yu, F., Zhang, H., Dai, Z., Yuan, L., Lei, Z., Zhang, W., Zhu, K., et al. xRouteBench: A benchmark for LLM routing. https://huggingface.co/datasets/ulab-ai/xRouteBench, 2026a. Dataset introduced in "LLMRouter: Unified Infrastructure for Developing, Evaluating, and Deploying LLM Routers," arXiv:2608.06867.

Feng, T., Zhang, H., Lei, Z., Han, P., Patwary, M., Shoeybi, M., Catanzaro, B., and You, J. Fusionfactory: Fusing LLM capabilities with multi-LLM log data. *Transactions on Machine Learning Research (TMLR)*, 2026b. Dataset: https://huggingface.co/datasets/ulab-ai/FusionBench.

Fu, T., Min, Z., Zhang, H., Yan, J., Dai, G., Ouyang, W., and Wang, Y. Cache-to-cache: Direct semantic communication between large language models. In *ICLR*, 2026.

Gao, B., He, Z., Sharma, P., Kang, Q., Jevdjic, D., Deng, J., Yang, X., Yu, Z., and Zuo, P. Cost-efficient large language model serving for multi-turn conversations with CachedAttention. In *USENIX ATC*, 2024.

Han, M., Zhang, H., Chen, R., and Chen, H. REEF: Microsecond-scale preemption for concurrent GPU-accelerated DNN inferences. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2022. URL https://www.usenix.org/conference/osdi22/presentation/han.

Heo, T., Shafipour, R., Zhao, R., Golub, M., Kamani, M. M., Borkar, R., Chandran, M. T., Zardoshti, P., et al. Cross-model KV cache transfer in LLM families: A closed-form linear mapping for prefill reuse. *arXiv preprint arXiv:2608.03893*, 2026.

Jimenez, C. E., Yang, J., Wettig, A., Yao, S., Pei, K., Press, O., and Narasimhan, K. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations (ICLR)*, 2024.

Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J. E., Zhang, H., and Stoica, I. Efficient memory management for large language model serving with PagedAttention. In *SOSP*, 2023.

Kwon, Y. D., Williams, M., Li, R., Kouris, A., and Venieris, S. I. WhiFlash: Accelerating speculative decoding with token-level cross-paradigm routing. In *EMNLP*, 2026.

Leviathan, Y., Kalman, M., and Matias, Y. Fast inference from transformers via speculative decoding. In *ICML*, 2023.

Li, H., He, R., Mang, Q., Zhang, Q., Mao, H., Chen, X., Zhou, H., Zhang, H., et al. Continuum: Efficient and robust multi-turn LLM agent scheduling with KV cache time-to-live. *arXiv preprint arXiv:2511.02230*, 2025.

Liu, B., Qiu, H., Goiri, Í., Fonseca, R., Bianchini, R., and Choukse, E. GitHub copilot coding agent dataset 2026. https://github.com/Azure/AzurePublicDataset/blob/master/GitHubCopilotCodingAgentDataset2026.md, 2026a. Companion paper: "Agentic Coding in the Wild: Characterizing GitHub Copilot Traces at Production Scale," arXiv:2608.00101.

Liu, B., Qiu, H., Goiri, Í., Fonseca, R., Bianchini, R., and Choukse, E. Agentic coding in the wild: Characterizing GitHub Copilot traces at production scale. *arXiv preprint arXiv:2608.00101*, 2026b.

Liu, Y., Huang, Y., Yao, J., Feng, S., Gu, Z., Du, K., Li, H., Cheng, Y., et al. DroidSpeak: KV cache sharing for cross-LLM communication and multi-LLM serving. *arXiv preprint arXiv:2411.02820*, 2024.

Liu, Y., Cheng, Y., Yao, J., An, Y., Chen, X., Feng, S., Huang, Y., Shen, S., et al. LMCache: An efficient KV cache layer for enterprise-scale LLM inference. *arXiv preprint arXiv:2510.09665*, 2025.

Narasimhan, H., Jitkrittum, W., Rawat, A. S., Kim, S., Gupta, N., Menon, A. K., and Kumar, S. Faster cascades via speculative decoding. In *ICLR*, 2025.

Ong, I., Almahairi, A., Wu, V., Chiang, W.-L., Wu, T., Gonzalez, J. E., Kadous, M. W., and Stoica, I. RouteLLM: Learning to route LLMs with preference data. *arXiv preprint arXiv:2406.18665*, 2024.

OpenAI. Introducing GPT-5. https://openai.com/index/introducing-gpt-5/, 2025.

Pan, Z., Patel, A., Hu, Z., Shen, Y., Guan, Y., Li, W.-L., Qin, L., Wang, Y., et al. KVFlow: Efficient prefix caching for accelerating LLM-based multi-agent workflows. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 38, 2025. URL https://papers.nips.cc/paper_files/paper/2025/hash/b7971d31a7d5eb0f1eed2f8f6f368195-Abstract-Conference.html.

Patel, P., Choukse, E., Zhang, C., Shah, A., Goiri, Í., Maleki, S., and Bianchini, R. Splitwise: Efficient generative LLM inference using phase splitting. In *ISCA*, 2024.

Paul, R. K. and Nandy, S. AgentRouter: Heterogeneous model routing for cost-optimal multi-step agentic workflows. In *ICML Workshops*, 2026.

Qiao, Y., Anzai, S., Yu, S., Ma, H., Yang, S., Wang, Y., Kim, M., Wu, Y., et al. ConServe: Fine-grained GPU harvesting for LLM online and offline co-serving. In *Proceedings of the 43rd International Conference on Machine Learning (ICML)*, 2026. URL https://proceedings.mlr.press/v306/qiao26i.html. Originally released as arXiv:2410.01228 (2024) under the title "ConServe: Harvesting GPUs for Low-Latency and High-Throughput Large Language Model Serving".

Qin, R., Li, Z., He, W., Zhang, M., Wu, Y., Zheng, W., and Xu, X. Mooncake: A KVCache-centric disaggregated architecture for LLM serving. *arXiv preprint arXiv:2407.00079*, 2024.

Qiu, H., Biswas, A., Zhao, Z., Mohan, J., Khare, A., Choukse, E., Goiri, Í., Zhang, Z., Shen, H., Bansal, C., Ramjee, R., and Fonseca, R. ModServe: Modality- and stage-aware resource disaggregation for scalable multimodal model serving. In *Proceedings of the 2025 ACM Symposium on Cloud Computing (SoCC 2025)*, pp. 817–830, 2025. doi: 10.1145/3772052.3772254.

Stojkovic, J., Zhang, C., Goiri, Í., Torrellas, J., and Choukse, E. DynamoLLM: Designing LLM inference clusters for performance and energy efficiency. In *2025 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, 2025.

Strati, F., Ma, X., and Klimovic, A. Orion: Interference-aware, fine-grained GPU sharing for ML applications. In *Proceedings of the Nineteenth European Conference on Computer Systems (EuroSys)*, 2024. doi: 10.1145/3627703.3629578. URL https://anakli.inf.ethz.ch/papers/orion_eurosys24.pdf.

Wang, Y., Chen, Y., Li, Z., Kang, X., Fang, Y., Zhou, Y., Zheng, Y., Tang, Z., et al. BurstGPT: A real-world workload dataset to optimize LLM serving systems. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD)*, 2025. doi: 10.1145/3711896.3737413.

Woo, S., Kil, J., Kim, H., Kim, M., Kim, J., Seo, A., Lee, S., Jo, M., et al. ICaRus: Identical cache reuse for efficient multi model inference. In *ICLR*, 2026a.

Woo, S., Kim, H., Shim, S., Jo, M., Jeong, H., Lee, J., Kim, J., Lee, S., et al. PrefillShare: A shared prefill module for KV reuse in multi-LLM disaggregated serving. *arXiv preprint arXiv:2602.12029*, 2026b.

Yang, J., Lieret, K., Jimenez, C. E., Wettig, A., Khandpur, K., Zhang, Y., Hui, B., Press, O., Schmidt, L., and Yang, D. Swe-smith: Scaling data for software engineering agents, 2025. URL https://arxiv.org/abs/2504.21798.

Yu, L., Lin, J., and Li, J. Stateful large language model serving with pensieve. In *Proceedings of the Twentieth European Conference on Computer Systems (EuroSys)*, 2025. doi: 10.1145/3689031.3696086.

Zhang, H., Feng, T., and You, J. Router-r1: Teaching LLMs multi-round routing and aggregation via reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 38, 2025. URL https://openreview.net/forum?id=DWf4vroKWJ.

Zhang, J., Liu, X., Hu, Y., Niu, C., Zeng, H., Tang, S., Wu, F., and Chen, G. From myopic selection to long-horizon awareness: Sequential LLM routing for multi-turn dialogue. *arXiv preprint arXiv:2604.12385*, 2026a.

Zhang, R., Kim, C., Feng, S., Du, K., Liu, Y., Zhong, Y., Ching, C.-W., Jiang, J., et al. Learning agent execution for KV-cache management in agentic serving. *arXiv preprint arXiv:2608.14624*, 2026b.

Zhang, Y., Li, H., Wang, Z., Feng, S., Yang, X., Wang, D., Zhang, B., Bai, L., et al. MTRouter: Cost-aware multi-turn LLM routing with history–model joint embeddings. In *ACL*, 2026c.

Zheng, L., Yin, L., Xie, Z., Sun, C., Huang, J., Yu, C. H., Cao, S., Kozyrakis, C., et al. SGLang: Efficient execution of structured language model programs. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 37, 2024.

AI USAGE DISCLOSURE

Generative AI tools, including ChatGPT, Claude, and GitHub Copilot, were used during this work to support code development and debugging, literature review, and manuscript preparation. Those AI tools were primarily used to assisting with implementation, polishing the language, and making the presentation clearer. The authors formulated the research questions, designed the system and experimental methodology, and interpreted and validated the results. The authors also reviewed and verified all AI-assisted code and written content.

A RESULTS FOR ALL MODEL PAIRS

Table 3 lists the model pairs with their full names, and Figure 14 shows the main metrics for every pair. Over all 22 simulated pairs, Harvest-SLO+Host cuts LLM-call latency by 1.3–17.7% (mean 6.3%), switch-call latency by 3.5–21.4% (mean 11.4%), switch-call TTFT by 9.2–56.4% (mean 44.9%), and routed token cost by 4.3–46.1% (mean 29.5%). Every pair gains in all four metrics.

B SIMULATION DETAILS

Deployments. Each model runs on the smallest tensor-parallel degree at which its weights take at most half of HBM: 7–12B models and GPT-OSS-20B on one H100; Gemma-2-27B, GPT-OSS-120B, and Granite-34B on two; Llama-3.x-70B and Mixtral-8x7B on four; Mixtral-8x22B and Qwen3-Next-80B-A3B on eight.

Latency model in Vidur. An iteration contains prefill chunks (q new tokens on c cached tokens) and decodes. Per layer, GEMMs take the maximum of compute and weight-read time; prefill attention adds compute and KV reads; decode attention adds KV reads and a per-sequence cost; tensor parallelism adds two all-reduces. MoE layers use active parameters for compute and total parameters for weight reads (Agrawal et al., 2024a). All operator latencies are grounded in measurements on H100 GPUs, providing high-fidelity estimates of prefill and decode execution times.

Disaggregated cluster. Each model's instances split into prefill and decode instances; the split (mostly 3P:1D) is the one with the highest SLO-compliant capacity for each model and normal-traffic trace. A prefill instance runs prefill-only iterations; KV moves to the decode instance, and the transfer and decode-queue wait count toward TPOT. A decode instance keeps a session's context after each call, so the next transfer sends only the missing tokens. Load is set to ρ=0.7 of each cluster's own capacity, so colocated and disaggregated runs compare at the same relative load.

C ADDITIONAL SENSITIVITY STUDIES

We vary one parameter at a time on the four setups of Figure 9d–g (one pair per trace), with 3 seeds $\times$ 3 windows at ρ=0.7, and report the mean over the setups. Reductions are relative to No Shadow on the calls that complete under every compared method. When a parameter changes the environment No Shadow runs in (router, cluster, normal traffic, prefill chunk, P/D configuration, models per router), No Shadow is evaluated at the same setup for a fair comparison. Cluster size and tensor parallelism use capacities measured for that configuration, and load stays at ρ=0.7 of that capacity, so agentic and normal traffic scale with the cluster. At the defaults, Harvest-SLO and Harvest-SLO+Host reduce LLM-call latency by 3.3% and 5.0%, as in Figure 9.

Router switch rate and routing policy. A two-state Markov router replaces the dataset's choices and switches model at each step with probability $p \in [0.05, 0.5]$, giving 0.5 to 5.0 switches per session; the dataset routers give 3.2. More switches give shadows more calls to help: Harvest-SLO+Host's LLM-call reduction grows from -0.2% to 5.8% and Oracle+Host's from 4.1% to 10.9% (Figure 15a). The gain per switch call falls, from 13.4% to 10.8% for Harvest-SLO+Host and from 30.8% to 19.0% for Oracle+Host (Figure 15b): with frequent switches, the session left the target model only a few steps earlier, so No Shadow already finds more of the context cached. Re-routing the sessions with the accuracy-maximizing or random policy changes the switch rate (4.9 switches per session under random routing), and Harvest-SLO+Host reduces LLM-call latency by 5.5%, 5.0% (default), and 5.7% (Figure 15c).

Controller parameters. Of the three controller parameters, the iteration cap β matters most. At β=0.25 TPOT, few shadow chunks fit and Harvest-SLO gives -0.2% (Harvest-SLO+Host 1.6%); at β=1, Harvest-SLO gives 4.2% and Harvest-SLO+Host 6.0% (Figure 15d). The SLO margin m moves Harvest-SLO+Host only within 4.9–5.1% (Figure 15e): most in-flight requests are far ahead of their SLO, so β, not the slack, bounds the budget. A larger credit rate δ lets shadow chunks stretch iterations that carry routed calls: Harvest-SLO+Host falls from 5.4% at δ=0 to 4.0% at δ=0.1, and Harvest-SLO peaks at 3.6% (Figure 15f). Across all controller settings, normal-traffic SLO attainment differs from No Shadow by only -0.04 to 0.27 percentage points.

Host tier. The host tier needs only 32 GB per GPU: at 16 GB, Harvest-SLO+Host's switch-call reduction drops from 12.9% to 11.9% and Host Tier's from 2.4% to 1.3%, while 64, 128, and 256 GB give identical results (Figure 15g). PCIe bandwidth from 8 to 64 GB/s moves Harvest-SLO+Host only within 12.0–13.0% (Figure 15h); Oracle+Host stays at 23.0% throughout (at no switch cost regardless of PCIe).

Cluster scale. Harvest-SLO+Host reduces LLM-call latency by 5.3% with 2 instances per model and 6.2% with 16, and Oracle+Host by 9.6% and 11.7% (Figure 16a). Doubling each model's tensor-parallel degree shortens prefill, so a cold switch call costs less and every method gains less: Harvest-SLO+Host 3.5%, Harvest-SLO 2.9%, and Oracle+Host 8.0% (versus 10.0%; Figure 16b).

Normal traffic. With code or BurstGPT normal traffic instead of conversations, Harvest-SLO reduces LLM-call latency by 5.1% and 5.7% (versus 3.3%), and Harvest-SLO+Host by 5.9% and 6.5% (Figure 16c). Normal-traffic SLO attainment rises by at most 0.4 percentage points.

Table 3. The 22 simulated model pairs; Table 4 lists the two SWE-bench pairs of the GPU testbed. Small and large are by total parameters; Params gives total (active) parameters for MoE models. Label is the pair's name in the figures used in the main paper (e.g., Figure 5).

Trace	Small model	Large model	Params (B)	Label in figures
xRouteBench	GPT-OSS-20B	GPT-OSS-120B	20.9 (3.6) / 116.8 (5.1)	OSS-20B / OSS-120B
	GPT-OSS-20B	Llama-3.3-70B-Instruct	20.9 (3.6) / 70.6	OSS-20B / L3.3-70B
	Llama-3.3-70B-Instruct	GPT-OSS-120B	70.6 / 116.8 (5.1)	L3.3-70B / OSS-120B
	Qwen2.5-7B-Instruct	Llama-3.3-70B-Instruct	7.6 / 70.6	Q2.5-7B / L3.3-70B
	Qwen2.5-7B-Instruct	Qwen3-Next-80B-A3B-Instruct	7.6 / 81.3 (3.8)	Q2.5-7B / Q3N-80B
FusionBench	Llama-3.1-8B-Instruct	Llama-3.1-70B-Instruct	8.0 / 70.6	L3.1-8B / L3.1-70B
	Llama-3.1-8B-Instruct	Mistral-Nemo-12B-Instruct	8.0 / 12.2	L3.1-8B / Nemo-12B
	Mistral-7B-Instruct-v0.3	Llama-3.1-70B-Instruct	7.2 / 70.6	M-7B / L3.1-70B
	Mistral-7B-Instruct-v0.3	Mixtral-8x22B-Instruct-v0.1	7.2 / 140.6 (39.1)	M-7B / Mx-8x22B
	Qwen2.5-7B-Instruct	Llama-3.1-70B-Instruct	7.6 / 70.6	Q2.5-7B / L3.1-70B
Router-R1	Llama-3.1-70B-Instruct	Mixtral-8x22B-Instruct-v0.1	70.6 / 140.6 (39.1)	L3.1-70B / Mx-8x22B
	Llama-3.1-8B-Instruct	Llama-3.1-70B-Instruct	8.0 / 70.6	L3.1-8B / L3.1-70B
	Mistral-7B-Instruct-v0.3	Gemma-2-27B-IT	7.2 / 27.2	M-7B / G2-27B
	Mistral-7B-Instruct-v0.3	Llama-3.1-70B-Instruct	7.2 / 70.6	M-7B / L3.1-70B
	Qwen2.5-7B-Instruct	Llama-3.1-70B-Instruct	7.6 / 70.6	Q2.5-7B / L3.1-70B
	Qwen2.5-7B-Instruct	Mixtral-8x22B-Instruct-v0.1	7.6 / 140.6 (39.1)	Q2.5-7B / Mx-8x22B
GHCP-coding	Gemma-2-9B-IT	Granite-34B-Code-Instruct	9.2 / 33.7	G2-9B / Granite-34B
	Gemma-2-9B-IT	Mixtral-8x7B-Instruct-v0.1	9.2 / 46.7 (12.9)	G2-9B / Mx-8x7B
	Gemma-7B	Mixtral-8x7B-Instruct-v0.1	8.5 / 46.7 (12.9)	G-7B / Mx-8x7B
	Llama-3.1-8B-Instruct	Llama-3.1-70B-Instruct	8.0 / 70.6	L3.1-8B / L3.1-70B
	Mistral-7B-Instruct-v0.3	Mixtral-8x22B-Instruct-v0.1	7.2 / 140.6 (39.1)	M-7B / Mx-8x22B
	Qwen2.5-7B-Instruct	Llama-3.1-70B-Instruct	7.6 / 70.6	Q2.5-7B / L3.1-70B

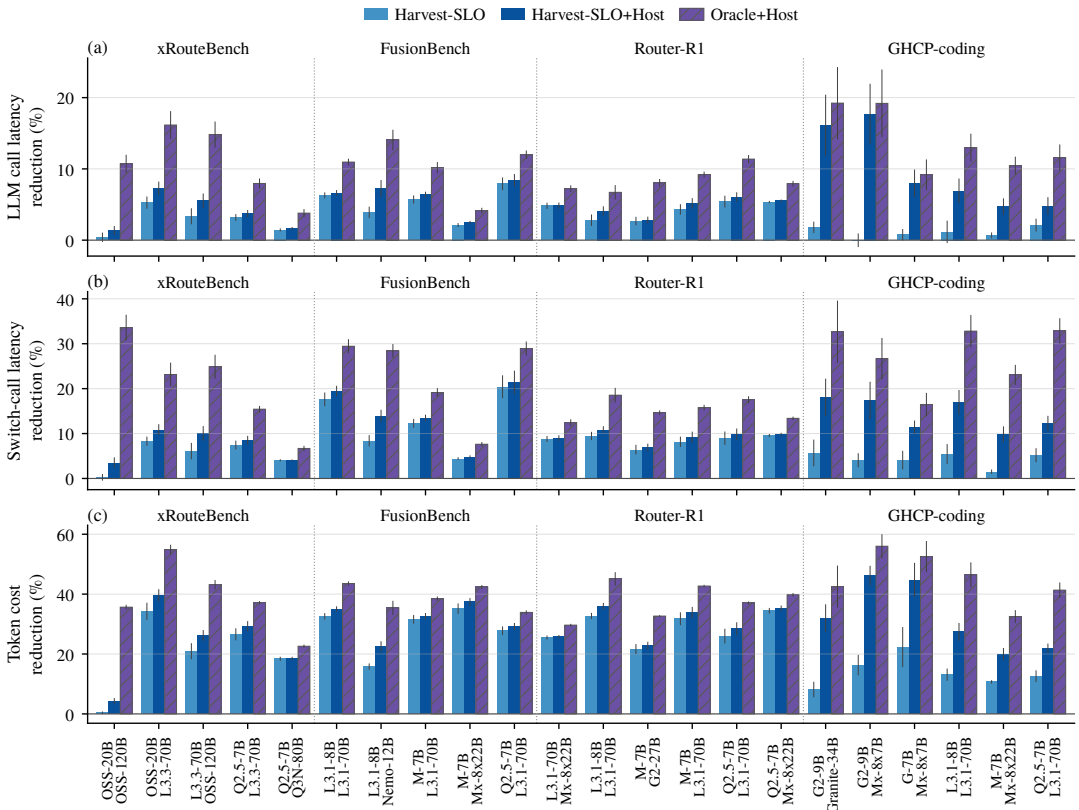

Figure 14. (a) LLM-call latency, (b) switch-call latency, and (c) routed token-cost reduction relative to No Shadow for all 22 simulated model pairs. Error bars: ±1 SE over 3 seeds × 3 windows.

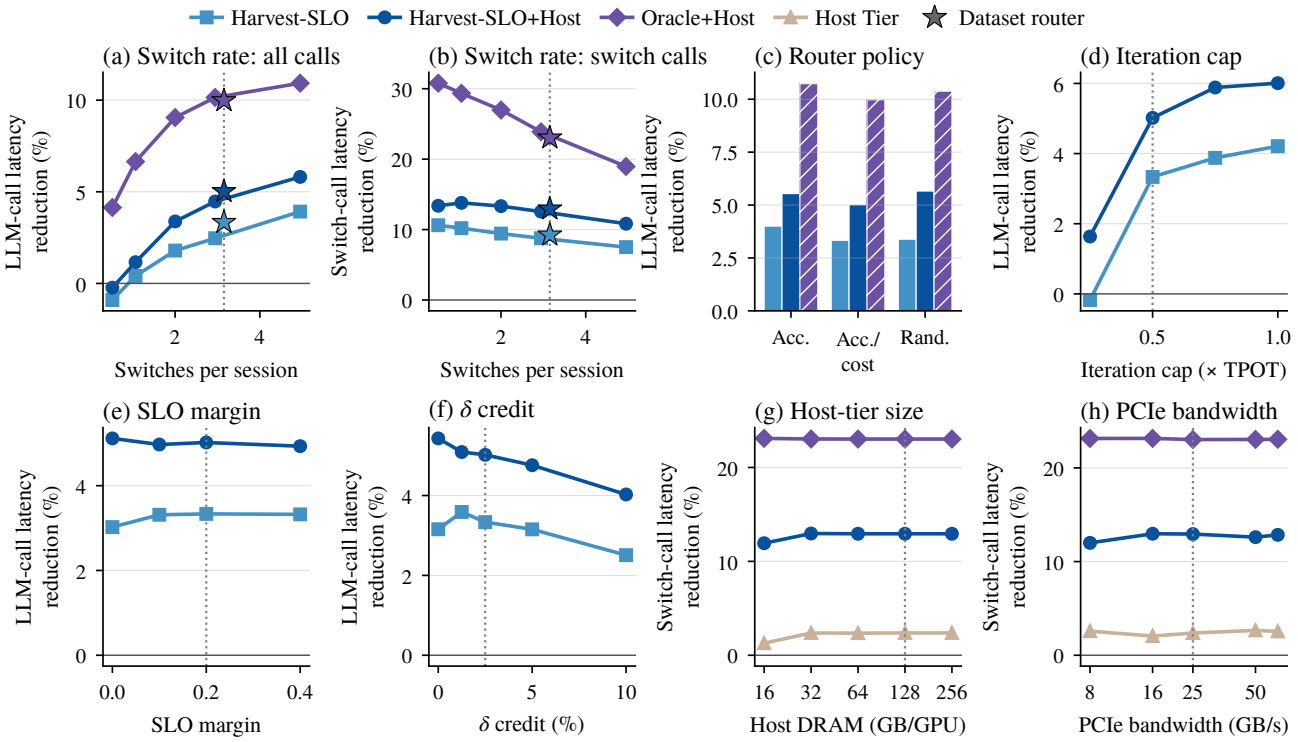

Figure 15. Sensitivity to the router and the controller. (a) LLM-call and (b) switch-call latency reduction versus switches per session under a Markov router (stars: dataset router). LLM-call latency reduction under (c) the max-accuracy, max-accuracy-per-cost (default), and random routing policies, and versus (d) the iteration cap β, (e) the SLO margin m, and (f) the credit rate δ. Switch-call latency reduction versus (g) host-tier size and (h) PCIe bandwidth. Mean over the four setups of Figure 9d–g; dotted lines mark the defaults.

More than two models per router. Each setup's pair grows along its trace's candidate models to N=3–5, and the sessions are re-routed by the accuracy-per-cost policy over all N models, raising switches per session from 3.2 to 5.1. With N>2, the next model of a switch is uncertain. Harvest-SLO+Host shadows the session to all other models, ordered by a leave-one-session-out prior on the next model; the top-1 variant shadows only to the most likely next model. Shadowing all targets reduces LLM-call latency by 6.3% at N=3 and 6.3% at N=5, versus 4.2% and 4.6% for top-1 (Figure 16d). The switch-call reduction of Harvest-SLO+Host falls from 12.9% at N=2 to 10.1% at N=5, and the share of its shadow tokens that go unused grows from 68.6% to 89.0% (top-1 at N=5: 73.0%; Figure 16e); energy per request rises by 2.8% (top-1: 0.1%). Oracle+Host grows from 10.0% to 14.6%, so better next-model prediction would recover more of the benefit from shadow prefill.

P/D disaggregation. With fewer prefill instances, prefill queues longer and a cold switch call costs more: Harvest-SLO reduces LLM-call latency by 8.7% with one prefill instance out of four and 4.9% with three (default), and Oracle by 11.7% and 7.3% (Figure 16f). KV-transfer bandwidth from 10 to 160 GB/s per GPU moves Harvest-SLO only within 4.9–5.4% (Figure 16g).

Chunk sizes. The prefill chunk budget matters little: Harvest-SLO+Host gives 4.9–5.1% from 1,024 to 8,192 tokens and 5.9% at 512 (Figure 16h). Shadow chunks below a minimum size (256 tokens) are deferred, since each chunk re-reads the whole cached context. This minimum matters more: at 64 tokens, Harvest-SLO+Host reaches 6.6%; at 512 and 1,024 tokens, chunks rarely fit in the budget and Harvest-SLO+Host drops to 2.8% and 1.4%.

D TESTBED DETAILS

D.1 Serving Protocol

Prototype. Each model uses one vLLM instance with prefix caching, chunked prefill, and priority scheduling (Table 4). After a call completes, the manager submits its prompt and response to the other model as lowest-priority shadow work with one output token. For non-append-only histories, it uses the next prompt's prefix, capped at the completed prompt length. The destination computes its own KV, without cross-model transfer. Newer completed calls supersede pending shadows. Admission requires an empty destination waiting queue and permits one in-flight shadow per instance. KV-usage and SLO-based admission do not constrain these runs. The prototype omits per-iteration credit, the SLO-slack controller, evict-first shadow KV, and the host tier.

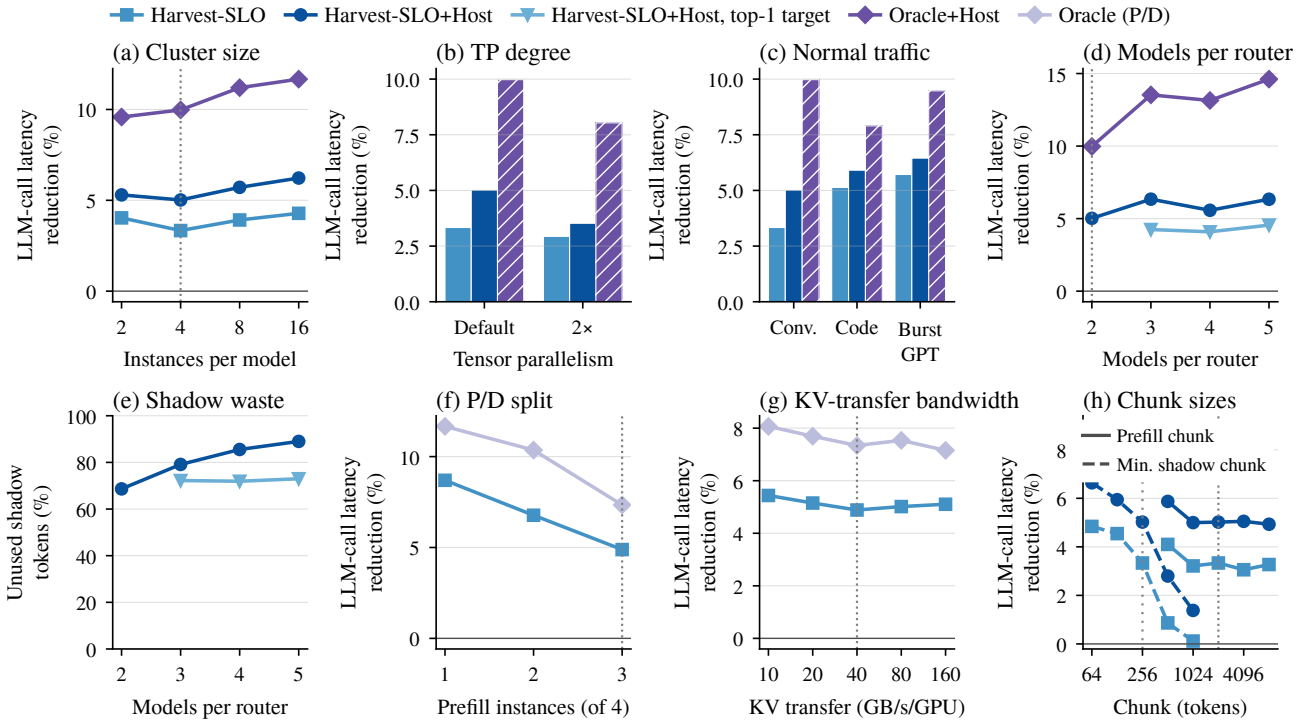

Figure 16. Sensitivity to the deployment. LLM-call latency reduction versus (a) instances per model, (b) tensor-parallel degree (default: Section B; 2×: twice that degree), (c) normal-traffic trace, and (d) models per router; (e) shadow tokens never used by a later call. In (d, e), Harvest-SLO+Host shadows the session to every other model; the top-1 variant shadows only to the most likely next model. P/D-disaggregated cluster: LLM-call latency reduction versus (f) prefill instances out of four per model and (g) KV-transfer bandwidth. (h) Prefill chunk budget (solid) and minimum shadow chunk (dashed). Mean over the four setups of Figure 9d–g; dotted lines mark the defaults.

Table 4. Testbed deployments and figure labels. Parentheses give H100-80GB GPU count and tensor-parallel degree (TP) per model instance.

Benchmark	Small model	Large model	Figure label
FusionBench	Qwen2.5-7B-Instruct (1 GPU, TP=1)	Llama-3.1-70B-Instruct (4 GPUs, TP=4)	Q7/L70
	Llama-3.1-8B-Instruct (1 GPU, TP=1)	Llama-3.1-70B-Instruct (4 GPUs, TP=4)	L8/L70
SWE-bench	SWE-agent-LM-7B (1 GPU, TP=1)	SWE-agent-LM-32B (2 GPUs, TP=2)	S7/S32
	SWE-agent-LM-7B (1 GPU, TP=1)	Qwen2.5-72B-Instruct (4 GPUs, TP=4)	S7/Q72

Traffic and replay. Normal traffic follows Poisson arrivals at ρ times each instance's calibrated capacity. Capacity is the highest tested rate with at least 90% attainment of both TTFT and TPOT SLOs. Their bounds are derived as $\max(1\,\mathrm{s}, 5t_{\mathrm{prefill}}(p))$ and $\max(25\,\mathrm{ms}, 5t_{\mathrm{decode}})$, using isolated prefill and batch-1 decode latencies. Capacities are 12.8, 11.0, 3.8, 13.3, 4.8, and 3.8 requests/s for Qwen2.5-7B, Llama-3.1-8B, Llama-3.1-70B, SWE-agent-LM-7B, SWE-agent-LM-32B, and Qwen2.5-72B, respectively. Azure traces (Stojkovic et al., 2025) supply input and output lengths capped at 8,192 and 1,024 tokens. FusionBench and SWE-bench session-arrival rates are 0.05/s and 0.01/s. SWE-bench samples with replacement from 30 S7/S32 and 95 S7/Q72 trajectories. Replay fixes routes, prompt structure, output lengths, and tool observations. Each call follows predecessor completion plus a lognormal tool gap with median 2 s. Append-only histories include replay-generated responses.

Measurement. FusionBench and SWE-bench runs last 900 s and 1,320 s, excluding the first 120 s. We match calls by session and step. Cache hit rate is cached input divided by total input tokens. LLM time per task excludes tool execution and uses the same matched latency sums as the all-call reduction. NVML measures total serving-GPU energy, including idle power and normal traffic, divided by the number of SLO-compliant requests.

Table 5. Replay workload under No Shadow at ρ=0.3. Lengths are tokens per call.

Model pair	Switch calls	Switches per session	Mean prompt	p99 prompt	Mean output	Hit rate (%)
Q7/L70	20%	2.5	19K	75K	52	85.2
L8/L70	26%	3.2	19K	75K	97	85.0
S7/S32	43%	15.4	41K	93K	175	93.9
S7/Q72	13%	6.3	33K	68K	249	96.6

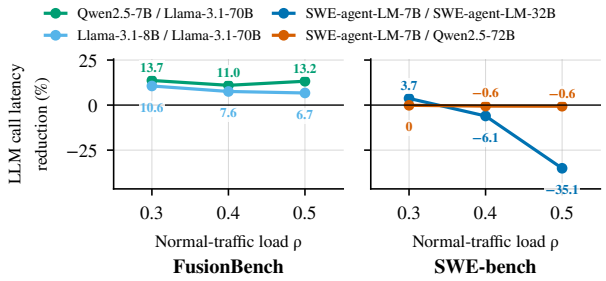

Figure 17. LLM-call latency reduction versus normal-traffic load at fixed session-arrival rates.

Workload dependence. Table 5 shows longer outputs for SWE-bench, which limit the task-time contribution of prefill savings. High baseline reuse does not eliminate switch-call prefill: SWE-bench switch calls cache 85.8% of input tokens on average, yet have 2.6× the same-model mean TTFT.

Admission under load. Figure 17 isolates the limit of queue-based admission. At ρ=0.5, both FusionBench pairs retain latency reductions, however S7/S32 slows by 35.1%. This is becasue an empty waiting queue can coexist with running decodes, allowing shadow work to extend their iterations. This motivates the iteration-level interference budget in Section 3.2.

D.2 Router Evaluation

Training. We train pair-specific bilinear matrix-factorization routers (Feng et al., 2025) on large-model trajectories from 50 tasks. Labels measure model agreement, not task success. S7/S32 uses teacher-forced greedy-token agreement thresholded at the training median. S7/Q72 uses agreement on tool calls and key arguments. Features combine embeddings of the last 4K prompt tokens, the latest action and observation, and compact history with standardized step features. Three-fold task-level cross-validation yields AUCs of 0.76 and 0.74, respectively. Training-cohort tasks use held-out-fold predictions, while other tasks use the fold ensemble.

Sensitivity. Figure 18 compares the default router, a last-4K-prompt-only variant, and offline action-match routing on 15 tasks per pair at ρ=0.3. Action-match chooses the small model when recorded actions agree. It requires both models' actions and is not deployable. Each policy's shadow and No Shadow calls are matched, so bars compare serving benefits rather than accuracy or absolute router latency. For S7/Q72, prompt-only routing improves switch-call TTFT more than the default (85.5% versus 65.9%), but switches less often (2.5% versus 13.0%). Its all-call TTFT reduction is therefore smaller (15.2% versus 17.5%). Switch frequency and per-switch benefit jointly determine workload-wide improvement.

Table 6. Resolved SWE-bench Verified tasks on the fixed 50-task cohort.

Configuration	Accuracy (%)
SWE-agent-LM-7B	16.0
SWE-agent-LM-32B	42.0
Qwen2.5-72B-Instruct	14.0
S7/S32 router	32.0
S7/Q72 router	14.0

D.3 Accuracy

Agent protocol. We execute SWE-agent with the SWE-smith tool configuration (Yang et al., 2025) and grade all 50 tasks with the official harness (Table 6). Unappliable patches count as unresolved. To preserve cumulative prefixes, we disable replacement of older tool observations with placeholders. SWE-bench models use YaRN with a 131,072-token context limit, without sliding-window history or silent truncation. Tool-output bounds are unchanged. We remove the monetary stopping budget and retain the 75-call limit.

E DISCUSSION

More than two models for model switches. A router can choose among any number of models. With N models, a session can switch to any of the other $N-1$, so SHADOWPREFILL must also guess which model comes next. SHADOWPREFILL ranks the candidates by a prior on the next model and spends shadow capacity on the most likely target first (Section 3.1). With five models, shadowing all

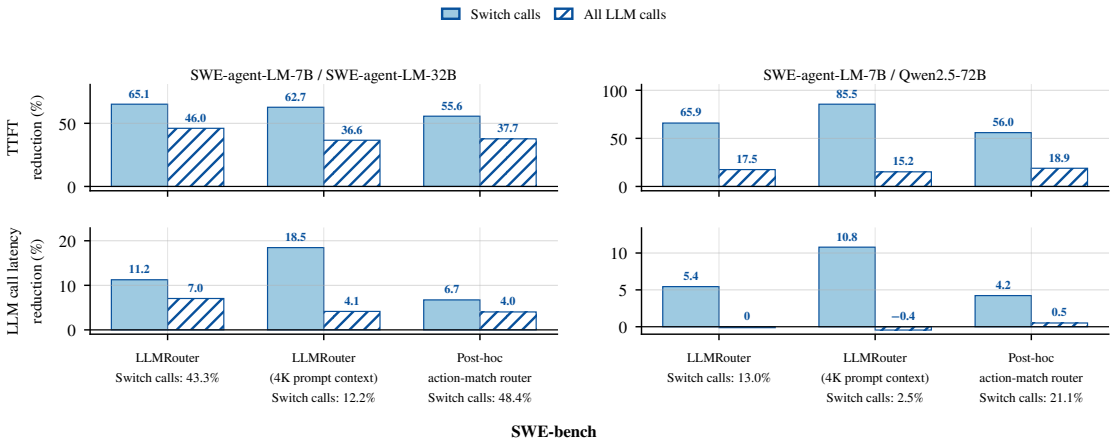

Figure 18. Shadow-prefill latency reductions relative to each router's own No Shadow baseline. Solid bars show switch calls. Hatched bars show all calls.

targets still reduces LLM-call latency by 6.3%, but 89.0% of shadow tokens go unused and energy per request rises by 2.8%; shadowing only the top-1 target gives 4.6% at 0.1% more energy (Section C). Meanwhile, Oracle+Host grows from 10.0% to 14.6%, because more models mean more switches. The gap between the two comes from next-model prediction, which a router can close by exposing its scores for upcoming calls, or which an agent's plan can supply when it fixes the model of the next steps ahead of time.

What does it mean for the router? Switch-aware routers penalize switches (Zhang et al., 2026a;c), because a switch has so far cost a cold prefill. With SHADOWPREFILL, the cost of a switch depends on how much of the candidate's prefix is warm, and the router can query that state and price each switch accordingly. A router that switches more freely gains accuracy (Section 5.3), and SHADOWPREFILL makes those switches cheaper: with frequent switches, Harvest-SLO+Host reduces LLM-call latency by up to 5.8% (Section C). Router and serving system can also cooperate in the other direction: the router's switch probabilities can drive the ordering of shadow jobs, and a more accurate predictor lets a gate such as Predictive skip unlikely targets while missing fewer switches.

Finer-grained model switching. SHADOWPREFILL targets routers that decide per LLM call. Systems that switch within a call, per token or per segment, such as speculative decoding (Leviathan et al., 2023; Chen et al., 2023a), speculative cascades (Narasimhan et al., 2025), and token-level routing (Kwon et al., 2026), leave no gap between the decision and the next model's use of the context. These systems keep both models' KV current at every step by construction, and pay for it on the critical path of decoding. SHADOWPREFILL instead trails the routed model by the output of the current call and runs only in harvested slack, which suits decisions made once per call. Cross-model KV translation (Liu et al., 2024; Fu et al., 2026; Heo et al., 2026) could fill the remaining gap at a switch for models in the same family, at the cost of approximate rather than exact KV.

When SHADOWPREFILL helps less. SHADOWPREFILL saves prefill, so its gain shrinks when prefill is a small part of a switch call. For targets above 50B, decoding dominates and switch-call latency falls by 3% (Section 5.5); doubling the tensor-parallel degree shortens prefill and lowers Harvest-SLO+Host's LLM-call gain to 3.5% (Section C). Long reasoning outputs, as from DeepSeek-R1, leave the target's instances little slack to harvest. Rare switches leave little to warm: at about 0.5 switches per session, Harvest-SLO+Host's LLM-call gain is -0.2%. Under heavy normal traffic, the SLO-slack controller finds little capacity (Harvest-SLO gives 0.7% at ρ=0.9), and the host tier provides most of Harvest-SLO+Host's gain.

Wasted work and energy. Shadow prefill is speculative: 63% of Harvest-SLO+Host's shadow tokens are never used, because the router decides only when the call arrives. On colocated clusters this work rides on iterations that run anyway, and energy per good request rises by 0.4%. On P/D-disaggregated clusters, shadow work wakes otherwise-idle prefill GPUs and raises energy by 11.5% (Section 5.4). Operators that cap power or bill energy can gate shadow work by switch probability, or limit it to the top-1 target, trading part of the latency gain for less waste.